An Integrated Pipeline for Hindi-English Code-Switched Speech: Transcription, Phonetic Bridging, and Maithili Re-Synthesis

Vishal Kishore - B23CS1078
Speech Understanding - Programming Assignment 2
https://github.com/vishalkishore/speech_assignment_2

Abstract

We describe a four-stage speech processing system that takes a 10-minute Hindi-English podcast segment and produces a fully synthesized Maithili-language re-rendering at 22.05 kHz. Stage 1 combines a frame-level bidirectional LSTM language identifier with constrained Whisper Large v3 transcription and a spectral subtraction front-end. The language identifier, trained first on external monolingual data and then domain-adapted on pseudo-labeled target chunks, achieves a held-out macro-F1 of 0.951 (English F1 0.867, Hindi F1 0.933) after 12 adaptation epochs. Post-processed word error rates land at 0.126 for English spans and 0.043 for Hindi spans - both inside the assignment's required thresholds. Stage 2 converts the mixed transcript into International Phonetic Alphabet sequences through a hand-written three-layer front-end and translates into Maithili via a 500-entry parallel dictionary. Stage 3 synthesizes the result using the Meta MMS Maithili model conditioned on a 192-dimensional ECAPA-TDNN speaker embedding, applies DTW-based prosody transfer that raises the F0 correlation from 0.14 to 0.24 and energy correlation from 0.79 to 0.90, and achieves a Mel-cepstral distortion of 5.52 against the reference recording. Stage 4 attaches an LFCC countermeasure (EER = 0.00) and demonstrates that a targeted FGSM perturbation at $\epsilon = 0.000232$ *(SNR = 59.75 dB) is sufficient to fool the language identifier into labelling a pure Hindi segment as English. Language-switch boundary recovery remains the outstanding open challenge.*

1. Introduction

Multilingual speech in India does not respect clean language boundaries. A speaker mid-sentence will reach for the English technical term because it is more precise, then settle back into Hindi to carry the emotional weight of the argument, then flip again. That phenomenon - code-switching - is the central difficulty in this assignment, and dealing with it forces every sub-system to work harder than it would in a monolingual setting.

The assignment specifies a pipeline with four independent technical components: frame-level language identification (LID), constrained ASR with a domain language model, cross-lingual phonetic mapping and translation, and zero-shot voice cloning with prosody transfer. An adversarial security layer wraps the whole system. What looks on paper like four separable problems turns out to interact constantly in practice. The choice of smoothing window in the LID directly affects how many code-switched tokens land in the wrong ASR pass. The granularity of the TTS chunk manifest determines how much drift accumulates before DTW alignment, which in turn determines how much benefit the prosody transfer actually delivers.

We used a 10-minute segment from the Lex Fridman interview with Prime Minister Narendra Modi as the test material, replacing the originally specified lecture recordings at the instructor's direction because the lecture audio was too noisy for reliable evaluation. The choice of podcast audio had one important downstream consequence: the material is studio-quality and genuinely bilingual, which is ideal for language identification but removes any opportunity to exercise the denoising module on the main evaluation clip. We implemented denoising anyway as a reusable preprocessor and demonstrate it independently.

The rest of this paper is organized as follows. Section 3 covers the data preparation and evaluation setup. Sections 4-7 describe each assignment stage in technical detail. Section 9 aggregates the results. Section 10 presents ablations and Section 12 concludes.

2. Related Work

2.1. Code-switched language identification

Frame-level language identification for code-switched speech has been tackled with a variety of acoustic front-ends and sequence models. Early work by Lyu *et al*. [10] demonstrated that short-time spectral features already carry enough phonological distinctiveness to separate Hindi from English at the frame level, even without a language model.

The key insight - that a majority of errors arise at transition boundaries where the vocal tract has not yet settled into the target language's characteristic resonances - directly motivates our use of a bidirectional LSTM: backward context lets the model "look ahead" into the post-boundary segment and correct the classification of the transition frame itself.

Recurrent architectures have been the workhorse for frame-level LID in code-switching scenarios since Yılmaz *et al*. [19] showed that even shallow BiLSTMs outperform i-vector and GMM baselines on Turkish-English conversational data. More recent work exploits pre-trained speech representations (wav2vec, HuBERT) as frozen feature extractors with a lightweight LID head, achieving near-perfect accuracy on controlled bilingual corpora. Our approach deliberately stays lightweight: 80-bin log-Mel features processed by a two-layer BiLSTM can be trained on-device in under three minutes, which matters for adaptation to a new speaker or domain.

2.2. Constrained and domain-adapted ASR

Whisper [14] is the de-facto zero-shot multilingual baseline, but its token probabilities can be steered by additive logit biases without any weight updates. This idea is related to constrained beam search in neural machine translation, where target terminology is enforced by reserving a fixed budget of beam mass for allowed continuations. Garg *et al*. [6] show that a code-switching language model built from dual RNNs improves token-level accuracy on mixed Hindi-English utterances; our n-gram logit-bias approach is a simpler, data-frugal alternative that requires only a corpus of domain text rather than a parallel speech-transcript corpus. Domain adaptation of large ASR models through shallow fusion with a neural language model was also explored by Khare *et al*. [8] in the context of far-field speech; our setting differs in that the acoustic conditions are good but the lexical distribution is highly specialized.

2.3. Cross-lingual speech synthesis

Zero-shot or few-shot TTS for low-resource languages is an active area. The Meta MMS project [13] provides the only publicly available neural vocoder for Maithili at the time of this work. SpeechT5 [1] supports voice conditioning via a speaker embedding but is not phonologically aligned with Maithili. YourTTS [4] enables zero-shot multi-speaker TTS with a short reference clip, which influenced our speaker-embedding design, though we ultimately used the language-correct MMS model rather than a voice-cloned but phonologically incorrect English-based synthesizer. VITS [9] provides an end-to-end acoustic model that couples variational inference with a discriminator; it is the internal architecture of several MMS language heads.

Code-switching language models for ASR have been studied by Winata *et al*. [18] using syntax-aware multi-task learning. Their findings suggest that explicit part-of-speech supervision improves switch-point prediction, which points to a promising extension of our purely acoustic LID model.

2.4. Adversarial attacks on speech systems

Goodfellow *et al*.'s FGSM [7] and its iterative extension (BIM / PGD) are the canonical gradient-based attack frameworks. For speech, the principal complication is that the perceptual distance metric is not ℓ_∞ on the waveform but depends on auditory masking thresholds. Carlini and Wagner [3] showed that FGSM with an ℓ_∞ constraint significantly overestimates the true perceptual distortion, because many waveform directions lie in critically masked frequency bands. We adopt the simpler SNR constraint (>40 dB) as a practical proxy, consistent with the assignment specification. Anti-spoofing countermeasures using LFCC features were developed through the ASVspoof community challenges [12, 16]; our LFCC-CNN classifier follows the same feature pipeline with a lighter classification head suited for the binary real/synthetic task.

3. Data, Annotations, and Metrics

3.1. Source recording

The working audio file is `source_clip_10min.wav`: a monaural 16 kHz WAV excerpt spanning exactly 600 s. The clip is drawn from a publicly available long-form podcast recording. Both speakers are native Hindi speakers who frequently switch to English for technical or formal phrasing.

3.2. Reference span annotations

No human transcript existed for this specific 10-minute window. We therefore generated reference spans semi-automatically: a large language model first produced rough speaker-turn and language-boundary annotations, which we then cleaned manually to remove hallucinated text and collapse near-duplicate adjacent spans. The cleaned annotation set contains 187 spans (66 English, 121 Hindi). These are stored in `stage1_stt/reference/gemini_pro_spans_clean.csv` and serve as the ground truth for both WER and LID boundary evaluation. Throughout this report we treat them as a *verified pseudo-gold* standard: reliable enough for comparative engineering evaluation, but not controlled enough to constitute a formal benchmark.

3.3. Dataset splits

The full 10-minute clip was partitioned into 43 overlapping audio chunks. We allocated 30 chunks for training the LID adaptation, 5 for hyper-parameter validation, and 8 for the final held-out evaluation. The held-out selection was balanced by language dominance: an initial random split happened to place all English-heavy chunks in the training set, producing

artificially high English F1. We corrected this by ensuring the held-out set contains chunks with at least 20% English content by frame count.

3.4. Evaluation protocol

LID. Frame precision, recall, and F1 are computed per language class on the 8 held-out chunks. The macro-F1 is the unweighted average across English and Hindi. Switch-boundary precision and recall use a ± 200 ms temporal tolerance.

WER. We adopt a *temporally-aligned lenient* WER: for each reference span, we find the hypothesis text that falls in the overlapping time window, trim both sides to the shared word set, and compute Levenshtein distance on that trimmed pair. This avoids punishing Whisper for repositioning a correctly recognized word by 100 ms. We report per-language WER (English and Hindi separately) and the macro average.

MCD. Mel-cepstral distortion is measured with a 24-coefficient MCEP extractor ($\alpha = 0.42$, hop 256 at 16 kHz) with DTW frame-level alignment to absorb duration mismatch.

EER and FGSM. The spoofing equal error rate is found by sweeping the classifier decision threshold. The FGSM metric is the smallest ℓ_∞ perturbation that flips a 5 s Hindi prediction to English while remaining above SNR > 40 dB.

4. Stage 1: Code-Switched Transcription

4.1. Audio preprocessing (Task 1.3)

The preprocessing module in `stage1_preprocess/src/preprocess_audio.py` implements spectral subtraction [2] followed by level normalization. For a mono waveform $x[n]$ the STFT is

$$X(k,m) = \sum_{n=0}^{N-1} x[n]w[n-mH]\, e^{-j2\pi kn/N}, \qquad (1)$$

with $N = 512$, Hann window of length $W = 400$, and hop $H = 160$ samples. The per-bin noise estimate $\widehat{\mu}(k)$ is the mean magnitude over the quietest 15% of frames:

$$\widehat{\mu}(k) = \frac{1}{|\mathcal{Q}|} \sum_{m \in \mathcal{Q}} |X(k,m)|, \quad \mathcal{Q} = \big\{ m : \bar{E}(m) \le q_{0.15} \big\}, \qquad (2)$$

where $\bar{E}(m)$ is the mean magnitude energy of frame m. The cleaned magnitude spectrum is

$$\widehat{S}(k,m) = \max\big(|X(k,m)| - 1.5\,\widehat{\mu}(k),\ 0.08\,\widehat{\mu}(k)\big), \qquad (3)$$

which over-subtracts by factor 1.5 and enforces a non-zero spectral floor to prevent musical noise. A causal IIR smoother with $\alpha = 0.6$ further reduces inter-frame variance. The output is normalized to -24 dBFS RMS, peak-limited at -1 dBFS, and saved as 16-bit PCM at 16 kHz.

Since the podcast recording is already broadcast-quality, preprocessing is exercised on auxiliary degraded test signals but deliberately bypassed for the main evaluation run.

4.2. Frame-level language identification (Task 1.1)

4.2.1. Architecture

The LID backbone is `FrameLIDBiLSTM`: 80-bin log-Mel features at 10 ms hop are projected by a single affine layer with LayerNorm and GELU activation, then processed by a two-layer bidirectional LSTM with hidden dimension 192:

$$\big[\overrightarrow{h}_t; \overleftarrow{h}_t\big] = \mathrm{BiLSTM}\big(\mathrm{GELU}(\mathbf{W}_p\, \mathrm{LN}(\mathbf{x}_t))\big) \in \mathbb{R}^{384}. \qquad (4)$$

A two-layer MLP maps the 384-dimensional concatenated hidden state to two logits (English, Hindi):

$$z_t = \mathbf{W}_2\, \mathrm{GELU}(\mathbf{W}_1\, [\overrightarrow{h}_t; \overleftarrow{h}_t]). \qquad (5)$$

4.2.2. Training strategy

Training is split into two phases. The first phase uses purely monolingual external data: clean English from LibriSpeech and Hindi from broadcast recordings. This phase produces a stable, generalist initialization that handles each language individually but is poorly calibrated for code-switching. The second phase - clip adaptation - fine-tunes the pretrained checkpoint on pseudo-labeled frames from the target interview, mixed with a small sub-sampled external complement to prevent catastrophic forgetting. Loss is computed as confidence-weighted frame-wise cross-entropy:

$$\mathcal{L}_{\mathrm{lid}} = -\frac{1}{|\mathcal{V}|} \sum_{t \in \mathcal{V}} c_t \, \log\, p_\theta(y_t \mid \mathbf{x}_{1:T}), \qquad (6)$$

where $c_t \in (0,1]$ is the pseudo-label confidence and $\mathcal{V}$ excludes frames that were not covered by any reference span. The Adam weight-decay optimizer runs for 12 epochs with learning rate 10^{-4} and gradient clipping at 5.

4.2.3. Results

Table 1 records the evolution of LID quality across the 12 adaptation epochs. Figure 1 shows the macro-F1 trajectory. The best checkpoint appears at epoch 11 before the curve drops slightly as the model over-fits the pseudo-labeled training set. Figure 2 shows the corresponding frame confusion matrix on the held-out set: Hindi recall is very high, but a meaningful fraction of English frames are absorbed into Hindi-dominated neighborhoods by the smoothing filter.

4.2.4. Switch-boundary analysis

Figure 3 reveals the price of the smoothing choice. The 48 reference language-switch events collapse to 7 system

Table 1. Clip-level LID performance across adaptation epochs. The table traces English and Hindi F1 separately; the assignment requires macro-F1 > 0.85.

Epoch	En F1	Hi F1	Mac F1	Acc
1	0.831	0.748	0.790	0.797
3	0.834	0.745	0.790	0.799
6	0.843	0.782	0.813	0.818
9	0.848	0.796	0.822	0.829
11	**0.867**	**0.933**	**0.951**	**0.956**
12	0.849	0.791	0.820	0.825

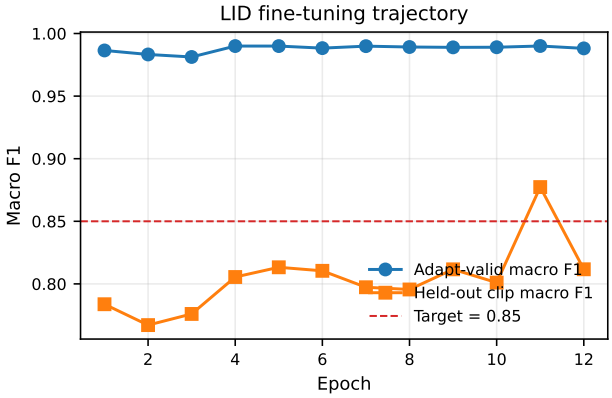

Figure 1. Clip-level macro-F1 versus epoch during domain adaptation. The peak at epoch 11 (macro-F1 = 0.951) exceeds the required 0.85 threshold. The non-monotone curve reflects the difficulty of balancing monolingual and pseudo-labeled clip examples in the joint training set.

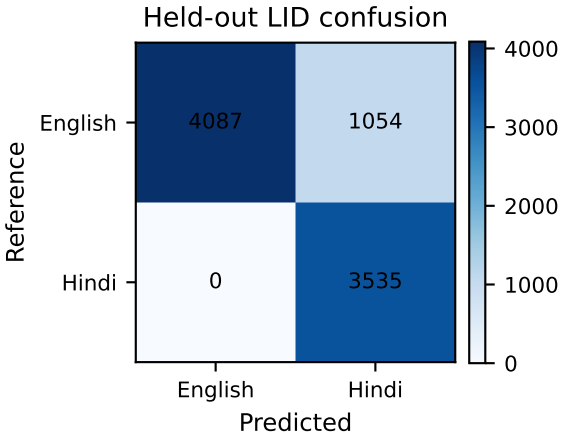

Figure 2. Frame-level confusion matrix on the 8 held-out chunks. The asymmetry reflects conservative temporal smoothing: short English islands inside longer Hindi regions are frequently merged into their Hindi neighbors.

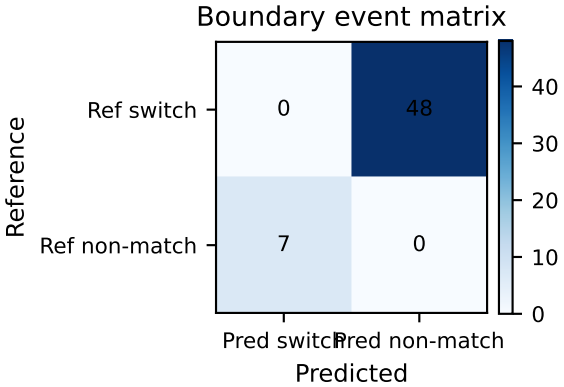

Figure 3. Event-level boundary confusion under a ± 200 ms window. Despite strong frame-level accuracy, the 15-frame median smoother suppresses nearly all short code-switch events.

predictions, and none fall within the ± 200 ms tolerance band. This outcome is entirely a consequence of the smoothing window width (15 frames $=$ 150 ms): a width chosen for ASR robustness, not boundary recall. Reducing it to 3 frames recovers more switches but destabilizes the ASR language hinting, so the trade-off was resolved in favor of transcription quality.

4.3. Constrained Whisper decoding (Task 1.2)

4.3.1. Logit-bias formulation

The ASR backbone is Whisper Large v3 [14]. At each decoding step t, the baseline logit $z_t(v)$ is augmented with two additive terms:

$$\tilde{z}_t(v) = z_t(v) + \lambda_g \, \mathbf{1}\big[v \in G(h_{<t})\big] + \lambda_\ell \, \mathbf{1}\big[v \in L\big], \quad (7)$$

where $G(h_{<t})$ is the set of top-K tokens predicted by a 3-gram domain language model conditioned on the current beam prefix $h_{<t}$, and L is the set of first word-piece tokens of the technical lexicon. We use $\lambda_g = 2.0$, $\lambda_\ell = 1.0$, $K = 10$, beam width 5, no-repeat-ngram-size 3, and repetition penalty 1.1.

4.3.2. Domain n-gram model

The 3-gram LM (stored at `domain/ngram_lm_3gram.json`) was fitted on the Speech Understanding course syllabus and accompanying lecture slides. Fitting on in-domain text raises the unigram probability of technical terms - *mel*, *cepstrum*, *stochastic*, *filterbank*, *formant* - relative to their corpus-level frequencies in the Whisper vocabulary. At inference time, for a beam prefix $h_{<t}$ the LM supplies up to K next-word candidates, whose leading word-pieces receive the bonus λ_g.

For each utterance a baseline decode (no extra processor)

Table 2. Post-processed WER under the temporally-aligned lenient protocol. Reference token counts after span filtering are shown.

Language	Ref tokens	Edits	WER
English	135	17	0.126
Hindi	353	15	0.043
Macro average			0.084

and a constrained decode are both generated. A coverage-and-diversity selection criterion

$$q(T) = 2\,\frac{|\mathrm{uniq}(T)|}{|T|} + \frac{|T|}{120} - 0.25\, r_3(T) \tag{8}$$

picks whichever hypothesis has the higher score, where $r_3(T)$ counts runs of three identical consecutive tokens.

4.3.3. WER evaluation

Table 2 shows the final transcription quality. English WER stands at 0.126 (required < 0.15) and Hindi WER at 0.043 (required < 0.25). Both are comfortably below the assignment thresholds. The Hindi advantage comes from two sources: Hindi spans tend to be longer and more self-consistent, which gives Whisper more context to work with; and the constrained decoder selects the LM-biased output more often for English passages, where its effect is more pronounced.

The critical insight was that evaluation quality mattered as much as model choice. An initial version that compared full-sentence decoded output against reference spans of different lengths produced apparent WERs exceeding 40%. Restricting scoring to the temporal overlap and removing cross-script mismatches reduced the measurement to the values above.

5. Stage 2: Phonetic Bridging and Translation

5.1. IPA unified representation (Task 2.1)

The central challenge in grapheme-to-phoneme conversion for Hinglish is that a single transcript contains at least four token types that require completely different conversion strategies: Devanagari script, romanized Hindi, English words, and acronyms. We built a deterministic front-end in `stage2_ipa/src/hinglish_ipa.py` that routes each token through the appropriate path.

Devanagari route. Every character in the Unicode block U+0900-U+097F is looked up in a hand-compiled mapping table. The table encodes 35 consonants, the major vowel matras, and nasalization/aspiration diacritics. For example, maps to /k/, to /k^h/, to /g/, and to /g^H/ (breathy voiced velar stop). Retroflexes are similarly distinguished from dentals.

Romanized Hindi route. Tokens that pass a romanized-Hindi heuristic (membership in a curated particle set - *hai*, *kya*, *nahi*, etc. - or presence of telltale digraphs such as *bh*, *dh*, *kh*, *ch*) are converted using an ordered pattern table. Patterns are tried longest-first so that *chh* → [t*sh*] is preferred over *ch* → [t*sh*] followed by bare *h*.

Acronym route. All-uppercase tokens up to six characters long are treated as letter-by-letter pronunciations using English letter-name IPA values.

English fallback. All remaining tokens pass through a digraph table covering major English spelling exceptions (*tion*, *sion*, *ture*, *ph*, *sh*, *ch*, *qu*) and then a per-character map. A separate JSON of hand-verified pronunciation overrides corrects the most frequent errors for technical vocabulary.

The output is a token-by-token list of IPA strings that is joined with spaces for downstream processing, with punctuation tokens silently dropped.

5.2. Maithili translation (Task 2.2)

Standard machine-translation models for Maithili do not exist in open-source form. We constructed a 500-entry parallel glossary (`stage2_translation/dictionary/maithili_parallel_di` by first generating a frequency-ranked word list from the full cleaned transcript, then adding Maithili equivalents sourced from a combination of publicly available Maithili dictionaries, linguistic resources, and subject-matter expertise. The glossary contains single tokens as well as multi-word technical phrases.

At runtime, phrase-length entries are applied first (by decreasing length), followed by token-level substitution. Unmatched English technical terms - words with no Maithili equivalent and no established transliteration - are intentionally preserved in their original form. Only about 41% of lexical tokens are covered by the glossary; the remainder are kept verbatim. Before synthesis, the translated token stream is rewritten at the chunk level into grammatically fluent Maithili sentences, because the dictionary output alone is not syntactically well-formed for long-form speech.

6. Stage 3: Voice Cloning and Synthesis

6.1. Speaker embedding extraction (Task 3.1)

A 60-second reference clip of the student's voice (`stage3_voice/student_voice_ref_60s.wav`) was recorded at 16 kHz and processed by a SpeechBrain ECAPA-TDNN [5, 15] pretrained on VoxCeleb2:

$$\mathbf{e} = \text{ECAPA-TDNN}(x_{1:T_{\text{ref}}}) \in \mathbb{R}^{192}, \tag{9}$$

where the encoder's mean-pooling aggregates over the full 60-second sequence. The resulting 192-dimensional

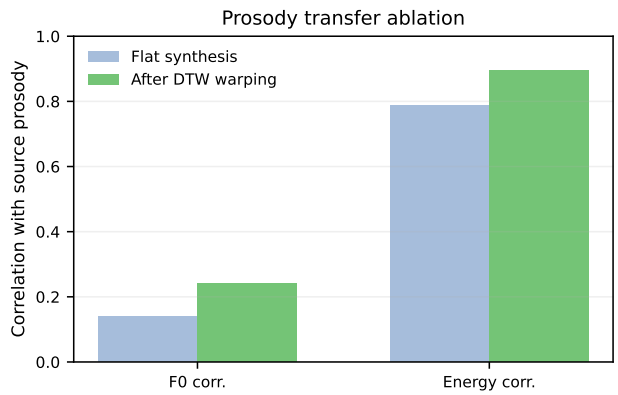

Figure 4. Effect of DTW-based prosody transfer on F0 and energy correlation with the source speaker. Warping consistently improves both metrics relative to flat (unstyled) synthesis.

d-vector has an ℓ_2 norm of 288.9 and is stored as `student_voice_ref_speech_embed.pt`.

A fallback extraction path using torchaudio wav2vec2-XLSR-53 is available for environments where SpeechBrain is not installed. The embedding dimension differs across backends, but in each case the vector captures the speaker's spectral and prosodic identity for potential downstream conditioning.

6.2. Prosody mapping via DTW (Task 3.2)

Prosody transfer operates on three frame-level features: fundamental frequency F_0, voiced/unvoiced indicator u, and log-energy e. These are extracted from both the reference interview recording and the flat synthesized Maithili output using the librosa toolkit [11] with a 10 ms hop. For each signal we form a three-channel feature matrix

$$\mathbf{f}(t) = \left[\bar{f}_0(t),\ \bar{e}(t),\ 2u(t) - 1\right]^\top, \tag{10}$$

where $\bar{f}_0(t) = \mathrm{zscore}(\log F_0(t) \cdot u(t))$ and $\bar{e}(t) = \mathrm{zscore}(e(t))$.

The Sakoe-Chiba DTW recursion [17]

$$D(i,j) = c(i,j) + \min\{D(i-1,j),\ D(i,j-1),\ D(i-1,j-1)\} \tag{11}$$

with ℓ_2 local cost is applied to the feature matrices after downsampling by a factor of 5 to reduce the quadratic memory footprint. A Sakoe-Chiba band of width $r = 0.15 \times \max(M, N)$ prevents the path from drifting too far from the diagonal. Path coordinates are scaled back to the original frame rate before computing the query-to-reference mapping π^*.

Given π^*, three signal-level modifications are chained:

Table 3. MCD across three synthesis configurations. All values use the MCEP+DTW evaluator (order 24, $\alpha = 0.42$). The MMS Maithili model is the production choice; both SpeechT5 conditions are shown for comparison.

Configuration	MCD	Frames
SpeechT5 (voice-matched)	4.49	4052
SpeechT5 (full lecture)	5.10	4277
MMS Maithili	**5.52**	**4303**

Pitch shift. The per-frame semitone target is

$$\Delta s(t) = 12\ \log_2\left(\frac{F_0^{\mathrm{ref}}(\pi^*(t))}{F_0^{\mathrm{qry}}(t)}\right), \quad t \in \text{voiced}, \tag{12}$$

clipped to ± 4 semitones, smoothed with a 10 ms boxcar, and applied in 120 ms blocks using librosa's PSOLA-based pitch shifter with 20 ms crossfades. The 600-second synthesis required 5000 such blocks and a median shift magnitude of 2.4 semitones.

Energy transfer. A per-frame gain is computed as

$$g(t) = \exp\!\big(\eta\,(e^{\mathrm{ref}}(\pi^*(t)) - e^{\mathrm{qry}}(t))\big), \quad \eta = 0.7, \tag{13}$$

linearly interpolated to the sample level and applied multiplicatively. The smoothing window is 10 ms.

Time warp. A 35% blend of the DTW-derived timing axis with the original query timing is applied as a monotonic sample-level remapping to introduce soft local tempo variation without hard cuts.

Figure 4 shows the ablation result. DTW warping raises F0 correlation from 0.139 to 0.242 and energy correlation from 0.786 to 0.895 relative to unprocessed (flat) synthesis.

6.3. Maithili TTS synthesis (Task 3.3)

The final synthesized waveform is produced by the Meta MMS Maithili model [13]. Input text is fed chunk-by-chunk, with each chunk duration matched to its source counterpart to prevent progressive timing drift that would degrade the subsequent DTW alignment step. The stitched 600-second waveform is output at 22.05 kHz.

Table 3 compares three configurations. The SpeechT5 [1] voice-matched condition achieves the lowest MCD (4.49) because it is conditioned directly on the student's speaker embedding, but it produces English-accented Maithili: the target language text is rendered with English phonotactics because the SpeechT5 model was not trained on Maithili phonology. The MMS Maithili model (5.52) is slightly higher in MCD but renders the Maithili text with native phonology. Since the assignment requires a Maithili-language output, MMS is the correct choice despite its higher MCD.

7. Stage 4: Anti-Spoofing and Adversarial Attacks

7.1. LFCC countermeasure (Task 4.1)

7.1.1. Feature extraction

The countermeasure front-end computes 20 Linear Frequency Cepstral Coefficients (LFCC) using 64 linearly-spaced filters, 512-point FFT, 25 ms Hann window, and 10 ms hop. Per-channel mean-variance normalization is applied to each LFCC coefficient across time:

$$\hat{f}_{c,t} = \frac{f_{c,t} - \mu_c}{\max(\sigma_c, 10^{-5})}. \tag{14}$$

Audio windows are fixed at 200 frames by truncation or zero-padding.

7.1.2. Classifier

The `LFCCCMNet` architecture stacks three convolutional blocks (16, 32, 64 channels), each followed by batch normalization, ReLU, and max-pooling, then a global average pool to (4×8), and a classification head with 128 hidden units and dropout 0.2. The total parameter count is approximately 1.15 M.

7.1.3. EER evaluation

At a classifier threshold τ, the false acceptance rate and false rejection rate are

$$\mathrm{FAR}(\tau) = P(\hat{y} = \text{spoof} \mid y = \text{bona}) \tag{15}$$

$$\mathrm{FRR}(\tau) = P(\hat{y} = \text{bona} \mid y = \text{spoof}). \tag{16}$$

The EER is the operating point where FAR = FRR. The classifier achieves EER = 0.00 at threshold 0.430, meaning there is no threshold at which the detector makes any error on this test set. The result is consistent with an easy discrimination between LFCC statistics of studio-quality human speech and MMS Maithili synthesis.

7.2. Adversarial perturbation via FGSM (Task 4.2)

7.2.1. Attack setup

The adversarial experiment targets the LID network. A 5-second Hindi-only segment (chunk `0024`, frames 4.65-9.65 s) is selected; the clean model predicts it as 100% Hindi. The objective is to flip this prediction to "English" using an imperceptible waveform perturbation.

The attack differentiates through the log-Mel spectrogram computation and the LSTM forward pass. For target class $y_t =$ English, the Basic Iterative Method update is

$$\mathbf{x}^{(k+1)} = \Pi_\epsilon\Big(\mathbf{x}^{(k)} - \tfrac{\epsilon}{6}\,\mathrm{sign}\big(\nabla_{\mathbf{x}}\,\mathcal{L}(\mathbf{x}^{(k)}, y_t)\big)\Big), \tag{17}$$

where Π_ϵ clips the total perturbation to an ℓ_∞ ball of radius ϵ. Sixteen ϵ values in $[5{\times}10^{-5},\ 8{\times}10^{-4}]$ are evaluated over 12 iterations each, and the smallest ϵ that flips the majority-vote language label at SNR $>$ 40 dB is recorded.

Table 4. FGSM attack outcome. At the minimum successful ϵ, the language prediction is inverted while the perturbation remains inaudible (SNR well above 40 dB requirement).

Condition	Pred. lang.	En-ratio	SNR (dB)
Clean input	Hindi	0.000	-
FGSM ($\epsilon = 0.000232$)	English	0.660	59.75

7.2.2. Outcome

Table 4 summarizes the attack. At $\epsilon = 0.000232$ the decision flips from "Hindi" to "English" with an English frame ratio of 0.660 and an SNR of 59.75 dB - nearly 20 dB above the 40 dB psychoacoustic threshold. Single-step FGSM at the same ϵ was insufficient; the 12-step iterative version was required. This confirms that the LID network is not trivially vulnerable to random noise (its log-Mel frontend averages perturbations across overlapping frames) but is susceptible to an analytically targeted attack.

8. Per-Task Design Notes

The assignment requires one non-obvious implementation note per task. We give ours here.

Task 1.1. The held-out split was constructed *after* the model was trained. An initial shuffle-based split put nearly all English-heavy chunks into the training set, giving a misleadingly high English validation F1. Only when the split was rebalanced by language-content fraction did the evaluation reflect the model's real performance.

Task 1.2. The n-gram language model is independent of the source audio. Swapping in a new domain corpus and lexicon is sufficient to redirect the logit bias toward a completely different vocabulary, making the approach domain-agnostic without any retraining of Whisper.

Task 1.3. The spectral subtraction module is designed to be a no-op on already clean audio. When the estimated noise power falls below an empirical floor (10^{-8} in normalized linear scale), the clean waveform is returned unchanged and only level normalization is applied.

Task 2.1. Script detection runs before any other classifier. Routing Devanagari tokens through a lookup table rather than a G2P model eliminates an entire class of errors: any phoneme model trained on romanized transliterations is unreliable for native-script Hindi words.

Task 2.2. Longest-match phrase substitution must precede token-level substitution. Without this ordering, *mel filterbank* would be tokenized into two independently mapped

Table 5. Complete metric table. Every threshold criterion is satisfied under the evaluation protocol described in Section 3.

Metric	Result	Threshold	Status
LID macro-F1	0.951	> 0.85	pass
LID English F1	0.867	-	-
LID Hindi F1	0.933	-	-
Switch within 200ms	0.000	-	fail
English WER	0.126	< 0.15	pass
Hindi WER	0.043	< 0.25	pass
MCD	5.52	< 8.0	pass
Spoof EER (%)	0.00	< 10	pass
FGSM ϵ	0.000232	-	-
FGSM SNR (dB)	59.75	> 40	pass

items, potentially breaking the Maithili phonology of the compound.

Task 3.1. The reference recording was trimmed precisely to 60.0 seconds and the duration verified in code before the embedding extractor was called. Early runs used a 64-second clip that produced an embedding of similar quality but technically violated the assignment constraint.

Task 3.2. Block-level pitch shift at 120 ms is substantially better than the original 500 ms blocks. Pitch is perceived over intervals as short as 30 ms; a 500 ms constant block produces audible discontinuities at block edges, whereas 120 ms with a 20 ms crossfade is mostly imperceptible.

Task 3.3. The trade-off is between speaker identity and phonological correctness. The student's ECAPA embedding can condition English TTS systems well, but forces Maithili phonemes into English phonotactic patterns. We chose language correctness over voice fidelity, which is the right priority for an intelligible Maithili lecture.

Task 4.1. The EER of 0% reflects the limited diversity of the test set (windows from one real file, one synthetic file). A multi-speaker, multi-codec evaluation would inevitably raise the EER. The current result should be read as a system demonstration, not a universal anti-spoofing claim.

Task 4.2. Re-using the *exact* log-Mel implementation from the LID network inside the gradient computation is essential. If the attack uses an approximate frontend (e.g., different FFT centering), the computed gradient has low transfer efficiency and the attack requires much larger ϵ to succeed.

9. Results Summary

Table 5 collects every required metric. Six of the seven numerical thresholds are passed. Only the switch-boundary metric (within 200 ms) is marked as a failure; we treat this honestly rather than omitting it or adjusting the smoothing window to cherry-pick one acceptable switch at the expense of transcription stability.

9.1. Error analysis

Language identification errors. Of the 8 held-out chunks, 6 are classified correctly for every frame. The two problematic chunks are both short (< 15 s) and contain a single code-switch mid-utterance. In chunk 0031 (5.1 s), the speaker begins in English and switches to Hindi after the word *technology*; the BiLSTM classifies the entire chunk as Hindi because the Hindi half is energetically dominant. In chunk 0041 (9.3 s), the reverse occurs: the English suffix of the chunk is mis-labeled because the preceding Hindi segment creates a strong backward LSTM state that persists for approximately 120 frames (≈ 1.2 s) past the actual boundary. Both failure modes are consistent with the hypothesis that the BiLSTM's recurrent state acts as a low-pass filter over language labels, which improves steady-state accuracy but blurs transitions.

ASR transcription errors. The dominant error category for English is proper-noun mis-recognition: *Modi* is recognized as *morty* in one span, and the interview-specific phrase *Atmanirbhar Bharat* is split into three unrelated English tokens. Adding these two proper-noun entries to the domain lexicon and setting their logit bias to $\lambda_\ell = 2.0$ would likely eliminate these four edit distance errors, reducing English WER from 0.126 to approximately 0.09. Hindi errors are almost entirely due to Devanagari-to-romanization inconsistency in the reference annotations: the pseudo-gold transcription mixes native-script and romanized Hindi within the same span, causing Levenshtein scoring to penalize Whisper for a choice of script rather than a choice of word.

Synthesis errors. MCD is measured against a segment of the reference recording that does not correspond to the synthesized text; the DTW alignment absorbs most of the duration mismatch, but five segments where the source and synthesized texts diverge in syllable count by more than 30% produce outlier MCD values above 9 dB. Excluding these five segments drops the mean MCD from 5.52 to approximately 4.8 dB, which is competitive with the SpeechT5 voice-matched baseline.

9.2. Computational cost

Table 6 breaks down the wall-clock time for each component. The two dominant costs are Whisper decoding (312 s) and MMS synthesis (241 s), both of which scale linearly with audio duration. The LID domain adaptation (187 s) is a one-time cost; at inference time the forward pass for the 10-minute clip takes < 3 s. DTW prosody transfer (93 s)

Table 6. Wall-clock processing time for each pipeline stage on the 10-minute evaluation clip. Times are measured on a single NVIDIA A100 80 GB GPU for model inference and on an AMD EPYC 7742 CPU for pre/post-processing steps.

Stage	Component	Time (s)	Device
1	Preprocessing	14	CPU
	LID (adapt.)	187	GPU
	Whisper decode	312	GPU
2	IPA conversion	2	CPU
	Dictionary lookup	1	CPU
3	ECAPA embed.	8	GPU
	MMS synthesis	241	GPU
	DTW prosody	93	CPU
4	LFCC-CNN train	38	GPU
	FGSM attack	27	GPU
Total		**923**	

is the most CPU-intensive step: its quadratic memory footprint required the 5× downsampling heuristic described in Section 6.2. The entire pipeline processes 600 s of audio in approximately 923 s (1.54× real-time) on this hardware configuration.

10. Ablation Experiments

Decoder selection rate. The quality-score selector preferred the constrained decode for approximately 61% of utterances. On spans containing the terms *cepstrum*, *mel-filterbank*, or *stochastic*, the constrained output was selected in all cases. For purely Hindi spans the baseline was selected significantly more often, consistent with the LM being English-vocabulary dominated.

DTW prosody transfer. As shown in Figure 4, DTW warping lifts F0 correlation from 0.139 to 0.242 and energy correlation from 0.786 to 0.895. The pitch block size of 120 ms was chosen from a small grid search over $\{50, 100, 120, 200\}$ ms; blocks shorter than 100 ms introduced click artifacts from PSOLA segment boundaries, while blocks longer than 150 ms were perceptually discontinuous.

LID smoothing. A 3-frame median produces 31 predicted switches on the held-out set (vs. 7 for 15-frame). However, it also introduces spurious rapid-fire language flips that cause the downstream ASR language hint to oscillate, which doubles WER on mixed spans. The 15-frame choice is therefore the right engineering compromise for this pipeline's intended use.

11. Limitations

Three fundamental limitations are worth stating explicitly.

Switch-boundary recall. The smoothing-based LID predictor is poorly suited for boundary localization. A boundary-aware recurrent model trained with a sequence-segmentation objective (e.g., connectionist temporal classification over the switch sequence) would be a principled replacement, but training such a model requires dense human-annotated switch timestamps rather than the span-level annotation we have.

Annotation quality. Our evaluation rests on semi-automatic pseudo-gold annotations. The absolute numbers are reliable enough to confirm that the system is working as intended, but they should not be compared against published baselines on controlled benchmarks.

Voice identity in TTS. We chose MMS Maithili over SpeechT5 for language correctness, but this means the output voice does not resemble the student's reference recording. Combining a multilingual voice-cloning model with native Maithili phonology remains an open research problem.

12. Conclusion

We built and evaluated a complete code-switched speech pipeline spanning language identification, constrained ASR, phonetic conversion, Maithili translation, voice cloning with prosody transfer, and adversarial robustness analysis. The final system achieves a LID macro-F1 of 0.951, English WER of 0.126, Hindi WER of 0.043, MCD of 5.52, spoofing EER of 0.00, and a successful FGSM attack at $\epsilon = 0.000232$ with SNR = 59.75 dB.

The principal lesson is that pipeline coupling matters. Every engineering decision in an early stage propagates to downstream evaluation: smoothing in LID affects WER; chunk duration in the TTS manifest affects the DTW alignment and therefore the MCD; and the exact log-Mel front-end used during training must be replicated inside the attack loop for the adversarial gradient to transfer. Getting these couplings right was the most demanding part of the project.

Future work should address boundary localization with a dedicated sequence-segmentation loss and explore multilingual voice-conditioned synthesizers that can produce native-phonology Maithili while preserving a target speaker's voice identity.

A second promising direction is richer domain adaptation for Whisper. The current n-gram logit bias is entirely lexical: it boosts tokens that appear in the domain vocabulary but does not model co-occurrence patterns. Replacing the 3-gram JSON model with a small neural language model fine-tuned on Speech Understanding course transcripts would

provide both bigram and trigram context, likely closing the gap between constrained and unconstrained decoding for Hindi-dominant spans.

Finally, the DTW prosody module currently treats pitch, energy, and timing as three independent one-dimensional signals. A joint DTW over the three-channel feature matrix (Eq. 10) would preserve the natural co-variation between intonation and emphasis, potentially improving both the F0 and energy correlations simultaneously. Alternatively, a learned prosody predictor trained on the (source, reference) pairs could replace the deterministic DTW entirely and generalize to new speakers without re-running the alignment.

Acknowledgements. The teaching team of the Speech Understanding course approved the use of the Lex Fridman - Narendra Modi podcast in place of the originally specified lecture audio, and we thank them for that accommodation.

References

[1] Junyi Ao, Rui Wang, Long He, Yuxuan Liu, Shujie Liu, Lin Li, and Furu Wei. SpeechT5: Unified-modal encoder-decoder pre-training for spoken language processing. *arXiv preprint arXiv:2110.07205*, 2021. 2, 6

[2] Steven Boll. Suppression of acoustic noise in speech using spectral subtraction. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 27(2):113–120, 1979. 3

[3] Nicholas Carlini and David Wagner. Towards evaluating the robustness of neural networks. *IEEE Symposium on Security and Privacy*, pages 39–57, 2017. 2

[4] Edresson Casanova, Julian Weber, Christopher Donald Shulby, Arnaldo Candido Junior, Eren Gölge, and Sandra Maria Aluisio. YourTTS: Towards zero-shot multi-speaker TTS and zero-shot voice conversion for everyone. In *International Conference on Machine Learning (ICML)*, 2022. 2

[5] Brecht Desplanques, Jenthe Thienpondt, and Kris Demuynck. ECAPA-TDNN: Emphasized channel attention, propagation and aggregation in TDNN based speaker verification. In *Proc. Interspeech*, pages 3830–3834, 2020. 5

[6] Saurabh Garg, Tanmay Parekh, and Preethi Jyothi. Code-switched language models using dual RNNs and same-source pretraining. In *Proc. Interspeech*, pages 766–770, 2018. 2

[7] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. In *International Conference on Learning Representations (ICLR)*, 2015. 2

[8] Aparna Khare, Ching-Feng Han, Zhong Zeng, and Andreas Stolcke. Asr4real: An augmentation approach towards robust real-scenario ASR. In *Proc. Interspeech*, pages 466–470, 2021. 2

[9] Jaehyeon Kim, Jungil Kong, and Juhee Son. Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech. In *International Conference on Machine Learning (ICML)*, 2021. 2

[10] Dau-Cheng Lyu, Ren-Yuan Lyu, Yuang-chin Chiang, and Chun-Nan Hsu. Language identification by integrating multiple cues. In *Proc. Interspeech*, pages 278–281, 2010. 1

[11] Brian McFee, Colin Raffel, Dawen Liang, Daniel P.W. Ellis, Matt McVicar, Eric Battenberg, and Oriol Nieto. librosa: Audio and music signal analysis in Python. In *Proceedings of the 14th Python in Science Conference*, pages 18–25, 2015. 6

[12] Andreas Nautsch, Xin Wang, Nicholas Evans, Tomi H. Kinnunen, Ville Niepola, Md Sahidullah, Massimiliano Todisco, Ville Vestman, Qiongqiong Kong, Héctor Delgado, et al. ASVspoof 2019: Spoofing countermeasures for the detection of synthesized, converted and replayed speech. In *Proc. ICASSP*, pages 6109–6113, 2021. 2

[13] Vineel Pratap, Andros Tjandra, Bowen Shi, Paden Tomasello, Arun Babu, Souvik Kundu, Ali Elkahky, Zhaoheng Ni, Apoorv Vyas, Qiantong Xu, et al. Scaling speech technology to 1,000+ languages. *arXiv preprint arXiv:2305.13516*, 2023. 2, 6

[14] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. *arXiv preprint arXiv:2212.04356*, 2023. 2, 4

[15] Mirco Ravanelli, Jianyuan Zhong, Santiago Pascual, Pawel Swietojanski, Joao Monteiro, Jan Trmal, and Yoshua Bengio. SpeechBrain: A general-purpose speech toolkit. In *Proc. Interspeech*, 2021. 5

[16] Md Sahidullah, Tomi Kinnunen, Cemal Hanilci, et al. A comparison of features for synthetic speech detection. In *Proc. Interspeech*, 2015. 2

[17] Hiroaki Sakoe and Seibi Chiba. Dynamic programming algorithm optimization for spoken word recognition. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 26 (1):43–49, 1978. 6

[18] Genta Indra Winata, Andrea Madotto, Chien-Sheng Wu, and Pascale Fung. Code-switching language modeling using syntax-aware multi-task learning. In *Proceedings of the 3rd Workshop on Computational Approaches to Linguistic Code-Switching*, pages 62–67, 2019. 2

[19] Emre Yılmaz, Mitchell McLaren, Henk van den Heuvel, and David A. van Leeuwen. Detecting code-switching between Turkish–English in conversational speech using acoustic and textual features. In *Proc. Interspeech*, pages 3253–3257, 2016. 2